

This living blueprint defines how Axiom-DSP should grow from a repository with strong tooling into an agentic audio-DSP engineering system. It starts from the current repo reality: Codex owns repository orchestration, Pi owns controlled execution, the user owns product direction and final listening acceptance, and external LLMs act only as advisory reviewers.

Core Idea

Axiom should eventually operate like a specialized audio-DSP engineering team, not like one default assistant session. The system should know who does what, how work is delegated, what evidence is required, and where human approval is mandatory.

The goal is not autonomous DSP rewriting. The goal is controlled collaboration:

- Codex manages the repository, plans work, protects source-of-truth docs, and prepares reviewable changes.
- Pi sessions execute constrained investigations, diagnostics, qualification, and candidate workflows through the Axiom harness.
- Specialist agents review DSP architecture, EEL2 safety, measurement quality, tooling, release readiness, and research claims from bounded roles.
- The user remains the final product authority, listening authority, and approval gate for publication, merge, and accepted-baseline promotion.
- Axiom Knowledge supplies research context, vocabulary, citations, and test design inspiration without becoming proof by itself.

Current State

Already implemented:

- Axiom Core accepted baseline: v4.1.4.11.
- Axiom Pi harness under tools/axiom-team/.
- Axiom Pi roles under tools/axiom-team/roles/.
- Axiom Pi slash commands through scripts/axiom_team.sh.
- Node harness commands through tools/axiom-team/bin/axiom-team.mjs.
- Codex operating guide, Pi runbooks, Labs policy, release gates, profile matrix, system status dashboard, issue templates, and session work-log PDF.
- Repo-tracked Axiom Codex skill source under tools/codex-skills/axiom-engineering/.
- Safe Codex helper CLI under tools/axiom-codex/.
- Dry-run-capable skill install helper at tools/install_axiom_codex_skill.py.
- Local-only Knowledge source-index schema and repo-safe source-note template.

Not implemented yet:

- Installed native Codex skill under ~/.codex/skills/axiom-engineering.
- Native Codex CLI slash commands outside the Pi harness.
- Autonomous Knowledge ingestion from private books or PDFs.
- Any agent permission to bypass user approval for candidate, publication, merge, or accepted-baseline promotion decisions.

Target Operating Model

The future Axiom agentic system should have four layers:

1. Human Authority Layer
2. Codex Orchestration Layer
3. Pi Execution Layer
4. Knowledge And Advisory Layer

Human Authority Layer

The user controls:

- product direction;
- listening acceptance;
- candidate approval;
- publication approval;
- merge approval;
- accepted-baseline promotion approval;
- whether Labs work should become candidate work.

No agent may bypass this layer for sound-changing work.

Codex Orchestration Layer

Codex should:

- read docs/system-status.md first;
- classify work as Core, Labs, Knowledge, Qualification, tooling, docs, or local-only;
- select the appropriate agent or Pi runbook;
- maintain the task ledger and session PDFs;
- update repository docs when state changes;
- prepare branches, commits, PRs, and reviews only after gates are satisfied.

Codex should not:

- silently create sound-changing candidates;
- merge broad experiments into Core;
- treat metrics as listening acceptance;
- use external research as proof without Axiom-specific evidence.

Pi Execution Layer

Pi should operate through the tracked Axiom harness, not uncontrolled shell automation. It should use:

- /axiom-doctor;
- /axiom-status;
- /axiom-corpus-status;
- /axiom-investigate;
- /axiom-hypothesis;
- /axiom-map-sub-gain;
- /axiom-stage-observability;
- /axiom-audit-stft;
- /axiom-audit-width-mono;
- /axiom-screen-exciter;
- /axiom-create-candidate;
- /axiom-qualify;
- /axiom-record-listening;
- /axiom-commit;
- /axiom-publish;
- /axiom-merge.

Real-JDSP measurements must remain serialized because the host audio route is shared mutable state.

Knowledge And Advisory Layer

Axiom Knowledge should contain:

- lawful bibliography notes;
- short summaries;
- concept maps;
- links to public sources;
- test-design questions;
- research-to-Axiom mappings.

Knowledge does not prove Axiom behavior. It helps agents ask better questions, design better diagnostics, and avoid shallow DSP reasoning.

External LLM feedback should enter through review triage:

text

claim -> safety filter -> destination -> evidence need -> issue/note/fixture/reject

Specialist Agent Map

Initial specialist set:

Role | Purpose | Primary Output

--- | --- | ---

Coordinator | Owns run flow, delegation, and decision summaries. | Work plan, role assignments, unresolved decisions.

DSP Architecture Lead | Reviews signal-chain intent and proposes falsifiable DSP hypotheses. | Affected stage, benefit, risk, listening target.

EEL Engineer | Reviews EEL2 implementation safety and minimal edit shape. | Safety findings, allowed edit boundary, static checks.

Measurement Engineer | Reviews whether tests can support the claimed conclusion. | Measurement plan, uncertainty, pass/fail meaning.

Qualification Lead | Decides whether evidence is strong enough for listening. | Gate status, missing evidence, listening eligibility.

Safety Auditor | Looks for regressions, scope creep, unsafe APIs, and policy violations. | Findings-first review.

Tooling Engineer | Maintains scripts, validators, reports, and repeatability. | Tool changes, tests, failure modes.

Signal Researcher | Connects research, prior evidence, and algorithm options. | Research summary, Axiom-specific questions.

Release Steward | Checks publication, PR, docs, policy hash, and merge readiness. | Release checklist and blockers.

Native Codex Extension Target

The v1 native Axiom Codex layer is repo-tracked but not auto-installed. It has:

- an Axiom skill source under tools/codex-skills/axiom-engineering/;
- role and handoff registries derived from existing Pi roles and repo policy;
- helper commands for common repo orchestration tasks;
- Knowledge lookup helpers for repo-safe notes plus optional local-only source index metadata;
- strict rules that mirror AGENTS.md, release gates, and local-artifact policy;
- clear handoff instructions for when Codex must start or delegate to Pi.

The v1 helper CLI is:

bash

python3 tools/axiom-codex/axiom_codex.py <command>

Supported commands:

- status-summary: summarize current baseline, active candidate, open investigations, and next safe actions.
- ready-check: run safe repo checks that do not execute real-JDSP workflows.
- agent-review: run a structured multi-role review scaffold from local role docs.
- knowledge-query: search Axiom Knowledge notes and return only cited, summarized context.

Native Codex slash commands are not assumed available in v1. Pi remains the slash-command runtime for Axiom execution.

Approval And Gate Rules

Mandatory user approval:

- creating a listening candidate after investigation;
- accepting a candidate after listening;
- publishing to GitHub;
- merging a PR;
- changing the accepted baseline policy;
- adding any new claim stronger than current evidence supports.

Mandatory stop conditions:

- accepted baseline hash mismatch;
- unscoped DSP rewrite;
- historical EEL file edited in place;
- normal-material clipping in qualification;
- real-JDSP route instability;
- private audio, captures, manifests, credentials, or source books entering git;
- external LLM review used as proof instead of advisory input.

Build Phases

Phase 1: Agent Registry And Blueprint

Create the durable docs and artifacts that define the system:

- this blueprint;
- a role registry;
- command registry;
- knowledge-use policy;
- approval matrix;
- handoff protocol between Codex and Pi.

Status: implemented for v1 through the blueprint, Codex skill references, and helper CLI.

Phase 2: Native Codex Skill Pack

Create a local Axiom Codex skill that can be installed under `~/.codex/skills/`. It should load Axiom-specific operating instructions, role summaries, and workflow shortcuts without replacing the existing repo gates.

Status: repo-tracked skill source implemented. Local install remains pending explicit user approval.

Phase 3: Knowledge Integration

Turn the curated Knowledge Base into structured, safe inputs:

- bibliography entries;

- concept summaries;
- source tags;
- Axiom-relevance notes;
- test-design questions.

The first version should be search-and-summarize only. Do not build autonomous claim generation.

Status: v1 schema, source-note template, and knowledge-query helper are implemented. Private source ingestion is not implemented.

Phase 4: Multi-Agent Review Workflow

Add repeatable multi-role reviews:

- architecture review;
- EEL safety review;
- measurement review;
- release review;
- external LLM triage review.

Each review should produce findings, evidence needs, and recommended next actions.

Phase 5: Operational Tightening

Connect the agent layer to repo maintenance:

- status dashboard refresh;
- session PDF updates;
- issue template generation;
- PR preparation;
- runbook selection;
- post-merge reconciliation.

Immediate Next Questions

Before implementation, decide:

- Whether to install the repo-tracked skill into `~/codex/skills/`.
- Whether to add deeper Knowledge ingestion for private PDFs/books after the local-only index is populated.
- Whether to convert the helper CLI outputs into richer machine-readable JSON reports.
- Whether to add native Codex slash-command support if the CLI harness exposes a stable command extension surface.